

Payout Destinations, Payables Modal & Promotion Edit Fix — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Close five gaps left after the already-implemented manual-payout system: bank fields dropped from the payables serializer, bank accounts never decrypted anywhere (making bank-only cafés unpayable), no payout-destination snapshot on [CafePayout](#), no owner-side payout-details editor, and the payables modal not using the shared [Modal](#) component — plus fix a confirmed one-line bug in promotion editing.

Architecture: Task 1 (promotion fix) is fully independent and can ship alone. Tasks 2–7 build the backend in dependency order: schema/model changes first, then repository logic (version bumping, snapshot population, atomicity), then the new API surface (reveal endpoint, owner payout-details endpoints), then the notification email. Tasks 8–10 build the frontend consumers of that API. Task 11 is full-suite regression verification.

Tech Stack: FastAPI + SQLAlchemy (async) + Alembic + pytest on the backend; Next.js App Router + TypeScript + React Query + Tailwind on the frontend.

Spec: [docs/superpowers/specs/2026-09-16-payout-payables-promotions-design.md](#)

Global Constraints

- No new admin role tier — all payout-destination visibility stays behind the existing single [admin](#) role (spec, "Out of scope").
- Bank account numbers must never be decrypted into any log, audit-log field, analytics event, or error message — only the reveal endpoint's direct JSON response may carry a decrypted value, and it must never be cached ([Cache-Control: no-store](#)).

- `OwnerAuditLog/AdminAuditLog` before/after content is masked representations only (e.g. `UPI: old@upi / Bank: ****1234 / IFSC HDFC0001234`) — never a decrypted account number or a password.
 - Historical `CafePayout` rows are never backfilled with a destination snapshot — pre-migration rows show "not recorded (pre-dates this feature)."
 - The legacy KYC//`setup` backend surface (`OwnerPayoutService`, `kyc_status`, `handle_kyc_webhook`) has a real live consumer (`payment_service.py:659`'s webhook handler, and `admin_service.py:733`'s verification-queue query) — do not delete `OwnerPayoutService`, `handle_kyc_webhook`, or any `kyc_status/razorpay_account_id` model field. Only the `/setup` and `/status routes` and the frontend `PayoutSetupCard` (whose only consumers are each other) are removed.
 - All financial writes in `create_payout()` (staleness check, `CafePayout` creation, snapshot population, `CafePayoutItem` creation, adjustment consumption) stay inside the existing single-transaction/single-commit structure — no early commits introduced.
-

Task 1: Fix promotion `None`-clearing bug

Files:

- Modify: `backend/app/repositories/promotion_repository.py:78-87`
- Test: Create `backend/tests/test_promotion_update.py`

Interfaces:

- Consumes: `PromotionRepository.update(promotion_id: UUID, update_data: dict[str, Any]) -> Optional[Promotion]` (existing signature, unchanged)
- Produces: nothing new consumed by later tasks — fully independent.
- ☐ **Step 1: Write the failing test**

```
# backend/tests/test_promotion_update.py
"""PromotionRepository.update() must apply an explicitly-cleared field
(value None, but present in update_data) – not silently skip it. Regression
test for the bug where clearing "Max Redemptions" or a KHELO code in the
owner UI appeared to succeed but reverted on refetch."""
import uuid
from datetime import datetime, timedelta, timezone
```

```

import pytest

from app.database import AsyncSessionLocal
from app.models.user import User, UserRole
from app.models.user_role import UserRoleMapping
from app.models.cafe import Cafe, VerificationStatus
from app.models.hardware_tier import HardwareTier
from app.models.promotion import Promotion
from app.core.security import get_password_hash
from app.repositories.promotion_repository import PromotionRepository

IST = timezone(timedelta(hours=5, minutes=30))

async def _make_cafe_with_promotion(db):
    owner = User(
        id=uuid.uuid4(), email=f"promo_upd_owner_{uuid.uuid4().hex[:8]}@test.com",
        full_name="Promo Update Owner",
        password_hash=get_password_hash("testpass123"),
        role=UserRole.CAFE_OWNER, is_active=True,
    )
    db.add(owner)
    await db.flush()
    db.add(UserRoleMapping(id=uuid.uuid4(), user_id=owner.id,
        role=UserRole.CAFE_OWNER))

    cafe = Cafe(
        id=uuid.uuid4(), owner_id=owner.id, name="Promo Update Café",
        address_line1="1 Test St",
        city="Bengaluru", state="Karnataka", pincode="560001",
        phone_number="+919876543210",
        verification_status=VerificationStatus.VERIFIED, is_active=True,
        opening_time=None, closing_time=None, bookable_stations=10,
    )
    db.add(cafe)

    now = datetime.now(timezone.utc)
    promo = Promotion(
        id=uuid.uuid4(), cafe_id=cafe.id, title="Weekday Special",
        discount_percentage=20, valid_from=now, valid_until=now +
        timedelta(days=30),
        days_of_week=[0, 1, 2, 3, 4], start_hour=10, end_hour=18,
        max_uses=5, khelo_code="WEEKDAY20",
    )
    db.add(promo)
    await db.commit()
    return owner, cafe, promo

@pytest.mark.asyncio
async def test_update_applies_explicit_none_to_clear_max_uses_and_khelo_code():
    async with AsyncSessionLocal() as db:
        _owner, _cafe, promo = await _make_cafe_with_promotion(db)
        repo = PromotionRepository(db)

        # Mirrors what PromotionUpdateRequest.model_dump(exclude_unset=True)
        # produces when the owner explicitly blanks these two fields in the
        # edit form – both keys are present with value None.

```

```

        updated = await repo.update(promo.id, {"max_uses": None, "khelo_code":
None})

        assert updated.max_uses is None
        assert updated.khelo_code is None

        refetched = await repo.get_by_id(promo.id)
        assert refetched.max_uses is None
        assert refetched.khelo_code is None

@pytest.mark.asyncio
async def test_update_still_ignores_fields_not_present_in_update_data():
    """A field genuinely absent from update_data (not sent by the frontend at
    all) must be left untouched – this is the exclude_unset=True contract
    the fix must preserve, not just "always overwrite everything"."""
    async with AsyncSessionLocal() as db:
        _owner, _cafe, promo = await _make_cafe_with_promotion(db)
        repo = PromotionRepository(db)

        updated = await repo.update(promo.id, {"title": "Renamed Special"})

        assert updated.title == "Renamed Special"
        assert updated.max_uses == 5
        assert updated.khelo_code == "WEEKDAY20"

```

- ❑ **Step 2: Run test to verify it fails**

Run: `cd backend && pytest tests/test_promotion_update.py -v` Expected:
`test_update_applies_explicit_none_to_clear_max_uses_and_khelo_code`
 FAILS — `assert updated.max_uses is None` fails because the old code left it at 5.
`test_update_still_ignores_fields_not_present_in_update_data` PASSES
 already (this behavior isn't broken).

- ❑ **Step 3: Fix the repository**

```
# backend/app/repositories/promotion_repository.py – replace lines 78-87
    async def update(self, promotion_id: UUID, update_data: dict[str, Any]) ->
Optional[Promotion]:
    promo = await self.get_by_id(promotion_id)
    if not promo:
        return None
    # update_data always comes from Pydantic's model_dump(exclude_unset=True)
    # – a key's mere presence here means the caller explicitly sent it,
    # including an explicit None meaning "clear this field". Filtering on
    # `value is not None` used to silently drop that clearing intent.
    for field, value in update_data.items():
        if hasattr(promo, field):
            setattr(promo, field, value)
    await self.db.commit()
    await self.db.refresh(promo)
    return promo
```

- ☐ **Step 4: Run test to verify it passes**

Run: `cd backend && pytest tests/test_promotion_update.py -v` Expected: both tests PASS.

- ☐ **Step 5: Run the full promotion test suite to check for regressions**

Run: `cd backend && pytest tests/test_khelo_promo_codes.py tests/test_launch_invariants_promotions.py -v` Expected: all PASS — no other test relies on the old drop-on-None behavior (confirmed: no other test exercises `update()` with a `None` value that was expected to be ignored while present in the dict).

- ☐ **Step 6: Commit**

```
git add backend/app/repositories/promotion_repository.py
backend/tests/test_promotion_update.py
git commit -m "fix(promotions): apply explicit None values when clearing fields on
edit"
```

PromotionRepository.update() silently skipped any field the owner explicitly cleared (Max Redemptions, KHELO code) because it filtered on 'value is not None' – but update_data already only contains explicitly-sent fields via `exclude_unset=True`, so a present None key is a deliberate clear, not an omission. The save looked successful but the old value reappeared on refetch."

Task 2: Migration + model changes for payout versioning and snapshot

Files:

- Create: `backend/migrations/versions/035_payout_destination_snapshot.py`
- Modify: `backend/app/models/owner_payout_account.py`
- Modify: `backend/app/models/cafe_payout.py`
- Create: `backend/app/models/owner_audit_log.py`

Interfaces:

- Produces: `OwnerPayoutAccount.version: int`;
`CafePayout.destination_type/destination_upi_vpa/destination_bank_account_masked/destination_bank_ifsc/destination_account_holder_name/destination_payout_account_id/destination_payout_account_version`;
`PayoutDestinationType` enum (`UPI="upi"`, `BANK="bank"`); `OwnerAuditLog` model (table `owner_audit_logs`) with fields `id`, `owner_id`, `action`, `entity_type`, `entity_id`, `before_summary`, `after_summary`, `created_at`.
All consumed by Tasks 3–7.

- ☐ **Step 1: Write the migration**

```
# backend/migrations/versions/035_payout_destination_snapshot.py
"""add owner_payout_accounts.version, cafe_payouts destination snapshot columns,
owner_audit_logs table

Revision ID: 035
Revises: 034
Create Date: 2026-09-16
"""

from alembic import op
import sqlalchemy as sa

revision = '035'
down_revision = '034'
branch_labels = None
depends_on = None

def upgrade():
    op.add_column(
        'owner_payout_accounts',
        sa.Column('version', sa.Integer(), nullable=False, server_default='1'),
    )

    op.add_column('cafe_payouts', sa.Column('destination_type', sa.String(10),
        nullable=True))
    op.add_column('cafe_payouts', sa.Column('destination_upi_vpa', sa.String(256),
        nullable=True))
    op.add_column('cafe_payouts', sa.Column('destination_bank_account_masked',
```

```

sa.String(20), nullable=True))
    op.add_column('cafe_payouts', sa.Column('destination_bank_ifsc', sa.String(20),
nullable=True))
    op.add_column('cafe_payouts', sa.Column('destination_account_holder_name',
sa.String(255), nullable=True))
    op.add_column(
        'cafe_payouts',
        sa.Column('destination_payout_account_id', sa.Uuid(),
sa.ForeignKey('owner_payout_accounts.id'), nullable=True),
    )
    op.add_column('cafe_payouts', sa.Column('destination_payout_account_version',
sa.Integer(), nullable=True))

    op.create_table(
        'owner_audit_logs',
        sa.Column('id', sa.Uuid(), primary_key=True),
        sa.Column('owner_id', sa.Uuid(), sa.ForeignKey('users.id'),
nullable=False),
        sa.Column('action', sa.String(100), nullable=False),
        sa.Column('entity_type', sa.String(50), nullable=False),
        sa.Column('entity_id', sa.String(255), nullable=False),
        sa.Column('before_summary', sa.Text(), nullable=True),
        sa.Column('after_summary', sa.Text(), nullable=True),
        sa.Column('created_at', sa.DateTime(timezone=True), nullable=False),
    )
    op.create_index('ix_owner_audit_logs_owner_id', 'owner_audit_logs',
['owner_id'])
    op.create_index('ix_owner_audit_logs_action', 'owner_audit_logs', ['action'])

def downgrade():
    op.drop_index('ix_owner_audit_logs_action', table_name='owner_audit_logs')
    op.drop_index('ix_owner_audit_logs_owner_id', table_name='owner_audit_logs')
    op.drop_table('owner_audit_logs')

    op.drop_column('cafe_payouts', 'destination_payout_account_version')
    op.drop_column('cafe_payouts', 'destination_payout_account_id')
    op.drop_column('cafe_payouts', 'destination_account_holder_name')
    op.drop_column('cafe_payouts', 'destination_bank_ifsc')
    op.drop_column('cafe_payouts', 'destination_bank_account_masked')
    op.drop_column('cafe_payouts', 'destination_upi_vpa')
    op.drop_column('cafe_payouts', 'destination_type')

    op.drop_column('owner_payout_accounts', 'version')

```

- ❑ **Step 2: Run the migration against the dev database**

Run: `cd backend && alembic upgrade head` Expected: migration 035 applies cleanly with no errors.

- ❑ **Step 3: Update the `OwnerPayoutAccount` model**

```
# backend/app/models/owner_payout_account.py – add this import and column
```

```
from sqlalchemy import String, DateTime, ForeignKey, JSON, Integer
```

Add the **version** column, right after **test_transfer_ref**:

```
test_transfer_ref: Mapped[str | None] = mapped_column(String(100),
nullable=True)
# Bumped by upsert_payout_details() whenever the destination (UPI or
# bank+IFSC) actually changes value – never on a no-op resubmit. Lets a
# CafePayout snapshot record exactly which version of this account was
# live when the payout was recorded (see
CafePayout.destination_payout_account_version).
version: Mapped[int] = mapped_column(Integer, default=1, nullable=False)
```

- ❑ **Step 4: Update the **CafePayout** model**

```
# backend/app/models/cafe_payout.py – add PayoutDestinationType enum after
CafePayoutStatus
class PayoutDestinationType(str, enum.Enum):
    UPI = "upi"
    BANK = "bank"
```

Add the import and the seven snapshot columns, right after **created_at**:

```
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True), default=lambda: datetime.now(timezone.utc),
nullable=False
)
# --- Payout destination snapshot ---
# Populated once, from the live OwnerPayoutAccount, at the moment this
# payout is created (see CafePayoutRepository.create_payout). Never
# updated afterward – a later change to the owner's OwnerPayoutAccount
# must never rewrite what this historical payout actually paid to.
# NULL on every row created before this feature shipped; the UI shows
# "not recorded (pre-dates this feature)" for those rather than
# backfilling a guess.
destination_type: Mapped[str | None] = mapped_column(
    Enum(PayoutDestinationType, values_callable=lambda x: [e.value for e in
x]),
    nullable=True,
)
destination_upi_vpa: Mapped[str | None] = mapped_column(String(256),
nullable=True)
destination_bank_account_masked: Mapped[str | None] = mapped_column(String(20),
nullable=True)
destination_bank_ifsc: Mapped[str | None] = mapped_column(String(20),
nullable=True)
destination_account_holder_name: Mapped[str | None] =
mapped_column(String(255), nullable=True)
destination_payout_account_id: Mapped[uuid.UUID | None] = mapped_column(
```

```

        ForeignKey("owner_payout_accounts.id"), nullable=True
    )
    destination_payout_account_version: Mapped[int | None] =
mapped_column(nullable=True)

```

- ❑ **Step 5: Create the `OwnerAuditLog` model**

```

# backend/app/models/owner_audit_log.py
import uuid
from datetime import datetime, timezone
from sqlalchemy import String, Text, DateTime, ForeignKey
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base

class OwnerAuditLog(Base):
    """Records sensitive actions an owner takes on their own account –
    starting with payout-detail changes. Deliberately separate from
    AdminAuditLog, which is admin-actor-shaped (admin_id/admin_email) and
    used for a different audience. before_summary/after_summary must
    always be masked representations – never a decrypted bank account
    number or a password."""
    __tablename__ = "owner_audit_logs"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    owner_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("users.id"),
    nullable=False, index=True)
    action: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    # e.g. "owner_payout_account"
    entity_type: Mapped[str] = mapped_column(String(50), nullable=False)
    entity_id: Mapped[str] = mapped_column(String(255), nullable=False)
    before_summary: Mapped[str | None] = mapped_column(Text, nullable=True)
    after_summary: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False,
        index=True,
    )

```

- ❑ **Step 6: Verify the models import cleanly and match the migration**

Run: `cd backend && python -c "from app.models.owner_payout_account import OwnerPayoutAccount; from app.models.cafe_payout import CafePayout, PayoutDestinationType; from app.models.owner_audit_log import OwnerAuditLog; print('OK')"` Expected: prints `OK` with no import errors.

- ❑ **Step 7: Commit**

```
git add backend/migrations/versions/035_payout_destination_snapshot.py
backend/app/models/owner_payout_account.py backend/app/models/cafe_payout.py
backend/app/models/owner_audit_log.py
git commit -m "feat(payouts): add version tracking and destination snapshot schema"
```

Adds `OwnerPayoutAccount.version` (bumped on real destination changes), seven destination-snapshot columns on `CafePayout` (populated once at payout creation, never updated after), and a new `OwnerAuditLog` table for owner-initiated sensitive actions. Purely additive migration, no backfill of historical `CafePayout` rows."

Task 3: Bump `OwnerPayoutAccount.version` on real destination changes

Files:

- Modify: `backend/app/repositories/owner_payout_repository.py:107-112`
- Test: Modify `backend/tests/test_owner_payout_repository_upsert.py`

Interfaces:

- Consumes: `OwnerPayoutAccount.version` (Task 2), the existing `destination_changed` boolean already computed inside `upsert_payout_details`.
- Produces: `upsert_payout_details()`'s returned `OwnerPayoutAccount.version` reflects exactly how many times the destination has actually changed — consumed by Task 4 (snapshot) and Task 7 (owner endpoint's `version` field in its response).
- ☐ **Step 1: Write the failing test**

Append to `backend/tests/test_owner_payout_repository_upsert.py`:

```
@pytest.mark.asyncio
async def test_version_starts_at_one_on_creation():
    async with AsyncSessionLocal() as db:
        owner = await _make_owner(db)
        repo = OwnerPayoutRepository(db)
        account = await repo.upsert_payout_details(
            owner_id=owner.id, upi_vpa="fresh@okaxis", bank_account_number=None,
            bank_ifsc=None, account_holder_name=None, bank_name=None,
            account_type=None, business_pan=None, default_holder_name=None,
        )
```

```
await db.commit()
assert account.version == 1
```

```
@pytest.mark.asyncio
```

```
async def test_version_bumps_when_destination_actually_changes():
    async with AsyncSessionLocal() as db:
        owner = await _make_owner(db)
        repo = OwnerPayoutRepository(db)
        await repo.upsert_payout_details(
            owner_id=owner.id, upi_vpa="v1@okaxis", bank_account_number=None,
            bank_ifsc=None, account_holder_name=None, bank_name=None,
            account_type=None, business_pan=None, default_holder_name=None,
        )
        await db.commit()

        updated = await repo.upsert_payout_details(
            owner_id=owner.id, upi_vpa="v2@okaxis", bank_account_number=None,
            bank_ifsc=None, account_holder_name=None, bank_name=None,
            account_type=None, business_pan=None, default_holder_name=None,
        )
        await db.commit()
        assert updated.version == 2
```

```
@pytest.mark.asyncio
```

```
async def test_version_does_not_bump_on_unchanged_resubmit():
    async with AsyncSessionLocal() as db:
        owner = await _make_owner(db)
        repo = OwnerPayoutRepository(db)
        await repo.upsert_payout_details(
            owner_id=owner.id, upi_vpa="stable_v@okaxis", bank_account_number=None,
            bank_ifsc=None, account_holder_name=None, bank_name=None,
            account_type=None, business_pan=None, default_holder_name=None,
        )
        await db.commit()

        # Same destination, only business_pan added – destination_changed
        # must be False, so version must not bump.
        updated = await repo.upsert_payout_details(
            owner_id=owner.id, upi_vpa="stable_v@okaxis", bank_account_number=None,
            bank_ifsc=None, account_holder_name=None, bank_name=None,
            account_type=None, business_pan="ABCDE1234F", default_holder_name=None,
        )
        await db.commit()
        assert updated.version == 1
```

- ❑ **Step 2: Run test to verify it fails**

Run: `cd backend && pytest tests/test_owner_payout_repository_upsert.py -v -k version` Expected: FAILS — `OwnerPayoutAccount` has no behavior bumping `version` yet, so all three `assert == 1/== 2` against a column that's either absent (if Task 2 wasn't run) or always `1`.

- ☐ **Step 3: Implement the version bump**

```
# backend/app/repositories/owner_payout_repository.py – replace lines 107-112
    if destination_changed:
        account.version += 1
        account.payout_verification_status = "unverified"
        account.verified_name = None
        account.verified_at = None
        account.verified_by_admin_id = None
        account.test_transfer_ref = None
```

- ☐ **Step 4: Run test to verify it passes**

Run: `cd backend && pytest tests/test_owner_payout_repository_upsert.py`

`-v` Expected: all PASS, including the three new tests and the four pre-existing ones.

- ☐ **Step 5: Commit**

```
git add backend/app/repositories/owner_payout_repository.py
backend/tests/test_owner_payout_repository_upsert.py
git commit -m "feat(payouts): bump OwnerPayoutAccount.version on real destination
changes"
```

Reuses the existing `destination_changed` detection (already used to reset `payout_verification_status`) to also increment version – only when the UPI ID or bank account/IFSC actually changes value, never on a no-op resubmit. This version is what CafePayout snapshots and what the admin payables flow checks for staleness between viewing and recording a payout."

Task 4: Snapshot destination + staleness check on `create_payout` (atomic)

Files:

- Modify: `backend/app/repositories/cafe_payout_repository.py`
- Modify: `backend/app/api/v1/admin_cafe_payouts.py`
- Modify: `backend/tests/test_cafe_payout_proof.py`
- Modify: `backend/tests/test_admin_cafe_payouts_api.py`
- Test: extend `backend/tests/test_cafe_payout_repository.py`

Interfaces:

- Consumes: `OwnerPayoutAccount.version` (Task 3), `PayoutDestinationType` (Task 2).
- Produces: `CafePayoutRepository.create_payout(..., destination_type: Optional[str] = None, expected_payout_account_version: Optional[int] = None)` — `destination_type` defaults to an inferred value ("upi" if the account has a UPI, else "bank") when not given, so every existing direct-repository test call keeps working unchanged; `expected_payout_account_version`, when given and mismatched against the live account, raises `ConflictException` (`error_code="PAYOUT_DESTINATION_STALE"`, HTTP 409) before any write. The API layer (this task, `admin_cafe_payouts.py`) makes both fields **required** on the actual admin-facing endpoint. Consumed by Task 9 (frontend `admin.ts`) and Task 10 (modal).
- ☐ **Step 1: Write the failing repository tests**

Append to `backend/tests/test_cafe_payout_repository.py`:

```
@pytest.mark.asyncio
async def test_create_payout_snapshots_destination_from_live_account(db_session):
    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe

    gamer = await _make_gamer(db_session, "snapshot_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=Decimal("500.00"))
    db_session.add(fee)
    cafe_row = (await db_session.execute(select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    account = OwnerPayoutAccount(
        id=uuid4(), owner_id=cafe_row.owner_id, upi_vpa="snap@okaxis",
        account_holder_name="Snap Holder", version=3,
    )
    db_session.add(account)
    await db_session.commit()

    repo = CafePayoutRepository(db_session)
    payout = await repo.create_payout(
        cafe_id=booking.cafe_id, admin_id=uuid4(),
        utr_reference="UTR-SNAP", payment_method="upi",
        destination_type="upi",
    )

    assert payout.destination_type.value == "upi"
    assert payout.destination_upi_vpa == "snap@okaxis"
    assert payout.destination_account_holder_name == "Snap Holder"
    assert payout.destination_payout_account_id == account.id
    assert payout.destination_payout_account_version == 3
```

```

@pytest.mark.asyncio
async def test_create_payout_infers_destination_type_when_not_given(db_session):
    """Existing/internal callers that don't pass destination_type explicitly
    (e.g. direct repository tests) still get a sensibly-populated snapshot."""
    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe

    gamer = await _make_gamer(db_session, "infer_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=Decimal("100.00"))
    db_session.add(fee)
    cafe_row = (await db_session.execute(select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(id=uuid4(), owner_id=cafe_row.owner_id,
upi_vpa="infer@okaxis"))
    await db_session.commit()

    repo = CafePayoutRepository(db_session)
    payout = await repo.create_payout(
        cafe_id=booking.cafe_id, admin_id=uuid4(),
        utr_reference="UTR-INFER", payment_method="upi",
    )
    assert payout.destination_type.value == "upi"

```

```

@pytest.mark.asyncio
async def test_create_payout_rejects_stale_expected_version(db_session):
    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe
    from app.core.exceptions import ConflictException

    gamer = await _make_gamer(db_session, "stale_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=Decimal("250.00"))
    db_session.add(fee)
    cafe_row = (await db_session.execute(select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(id=uuid4(), owner_id=cafe_row.owner_id,
upi_vpa="stale@okaxis", version=2))
    await db_session.commit()

    repo = CafePayoutRepository(db_session)
    with pytest.raises(ConflictException, match="changed since you opened"):
        await repo.create_payout(
            cafe_id=booking.cafe_id, admin_id=uuid4(),
            utr_reference="UTR-STALE", payment_method="upi",
            destination_type="upi", expected_payout_account_version=1,
        )

    # Nothing must have been written.
    remaining = await repo.get_outstanding_amount(booking.cafe_id)
    assert remaining == Decimal("250.00")

```

```

@pytest.mark.asyncio

```

```

async def test_create_payout_accepts_matching_expected_version(db_session):
    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe

    gamer = await _make_gamer(db_session, "fresh_version_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=Decimal("250.00"))
    db_session.add(fee)
    cafe_row = (await db_session.execute(select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(id=uuid4(), owner_id=cafe_row.owner_id,
upi_vpa="fresh@okaxis", version=2))
    await db_session.commit()

    repo = CafePayoutRepository(db_session)
    payout = await repo.create_payout(
        cafe_id=booking.cafe_id, admin_id=uuid4(),
        utr_reference="UTR-FRESH", payment_method="upi",
        destination_type="upi", expected_payout_account_version=2,
    )
    assert payout.destination_payout_account_version == 2

@pytest.mark.asyncio
async def test_create_payout_rolls_back_completely_on_partial_failure(db_session):
    """If anything fails after the CafePayout row is staged but before the
    transaction commits, nothing may be left half-written – no orphaned
    CafePayout row, no reduced outstanding balance."""
    from unittest.mock import patch
    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe
    from app.models.cafe_payout_item import CafePayoutItem

    gamer = await _make_gamer(db_session, "atomicity_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=Decimal("500.00"))
    db_session.add(fee)
    cafe_row = (await db_session.execute(select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(id=uuid4(), owner_id=cafe_row.owner_id,
upi_vpa="atomic@okaxis"))
    await db_session.commit()

    repo = CafePayoutRepository(db_session)
    original_add = db_session.add

    def _failing_add(instance):
        if isinstance(instance, CafePayoutItem):
            raise RuntimeError("simulated failure while adding payout items")
        return original_add(instance)

    with patch.object(db_session, "add", side_effect=_failing_add):
        with pytest.raises(RuntimeError, match="simulated failure"):
            await repo.create_payout(
                cafe_id=booking.cafe_id, admin_id=uuid4(),
                utr_reference="UTR-ATOMIC", payment_method="upi",
            )

```

```

await db_session.rollback()

remaining = await repo.get_outstanding_amount(booking.cafe_id)
assert remaining == Decimal("500.00")

leftover = (await db_session.execute(
    select(CafePayout).where(CafePayout.cafe_id == booking.cafe_id)
)).scalars().all()
assert leftover == []

```

Add `CafePayout` to this file's existing imports (top of `backend/tests/test_cafe_payout_repository.py`):

```

from app.models.cafe_payout import CafePayout, CafePayoutStatus

```

- ☐ **Step 2: Run tests to verify they fail**

Run: `cd backend && pytest tests/test_cafe_payout_repository.py -v -k "snapshot or infer_destination or stale or atomicity" Expected: FAIL — create_payout() doesn't accept destination_type/expected_payout_account_version yet, and CafePayout has no destination_* attributes.`

- ☐ **Step 3: Implement the snapshot + staleness check in `create_payout`**

```

# backend/app/repositories/caf  _payout_repository.py – update the imports at the
top
from app.core.exceptions import BadRequestException, ConflictException
from app.models.cafe_payout import CafePayout, CafePayoutStatus,
PayoutDestinationType

```

Replace the `create_payout` signature and the destination-check block:

```

async def create_payout(
    self,
    cafe_id: UUID,
    admin_id: UUID,
    utr_reference: str,
    payment_method: str,
    notes: Optional[str] = None,
    audit_log_data: Optional[dict] = None,
    proof_image_url: Optional[str] = None,
    admin_note: Optional[str] = None,
    paid_at: Optional[datetime] = None,
    destination_type: Optional[str] = None,

```

```
expected_payout_account_version: Optional[int] = None,
```

```
) -> CafePayout:
```

```
"""Create a CafePayout + its CafePayoutItem rows in a single transaction.
```

```
`destination_type` ("upi" or "bank") records the method actually
used for THIS payout – not the owner's stored preference, since an
OwnerPayoutAccount may hold both. If not given (internal/legacy
callers), it's inferred: "upi" if the account has a UPI, else
"bank". The real admin-facing API always supplies it explicitly.
```

```
`expected_payout_account_version`, when given, must match the live
OwnerPayoutAccount.version or a ConflictException (409,
PAYOUT_DESTINATION_STALE) is raised before any write – this is the
safeguard against an admin paying out a destination that changed
after they opened the payable but before they submitted. When not
given, the check is skipped (used by internal/legacy callers that
don't have a "version the admin last saw" to compare against).
```

```
When `audit_log_data` is provided, the AdminAuditLog entry for this
payout is added to the same session and committed atomically with the
payout/items – either all three persist, or none do. Callers that
don't need an audit trail (e.g. existing repository-level tests) can
omit it and behavior is unchanged.
```

```
Expected `audit_log_data` keys: admin_id, admin_email, and optionally
action, entity_type, entity_name, reason.
"""
```

```
import uuid as _uuid
```

```
cafe_row = (await self.db.execute(
    select(Cafe.owner_id, Cafe.payout_on_hold,
Cafe.payout_hold_reason).where(Cafe.id == cafe_id)
)).first()
if not cafe_row:
    raise BadRequestException("Café not found.")
owner_id, on_hold, hold_reason = cafe_row

if on_hold:
    raise BadRequestException(
        f"Payouts to this café are on hold: {hold_reason or 'no reason
given'}."
    )

payout_account = (await self.db.execute(
    select(OwnerPayoutAccount).where(OwnerPayoutAccount.owner_id ==
owner_id)
)).scalars().first()
has_upi = bool(payout_account and payout_account.upi_vpa)
has_bank = bool(
    payout_account
    and payout_account.bank_account_number_encrypted
    and payout_account.bank_ifsc
    and payout_account.account_holder_name
)
if not (has_upi or has_bank):
    raise BadRequestException(
        "This café hasn't added payout details yet – ask the owner to add a
UPI ID "
        "or bank account in Owner Settings before paying out."
```

```

    )

    if expected_payout_account_version is not None and payout_account.version
    != expected_payout_account_version:
        raise ConflictException(
            "Payout details changed since you opened this payable. Please
refresh and "
            "confirm the new destination before recording this payout.",
            error_code="PAYOUT_DESTINATION_STALE",
        )

    resolved_destination_type = destination_type or ("upi" if has_upi else
"bank")

    rows = await self.get_outstanding_fee_rows(caffe_id)
    adjustments = await self.get_unconsumed_adjustments(caffe_id)
    if not rows and not adjustments:
        raise BadRequestException("This café has no outstanding balance to pay
out.")

    fee_sum = sum((Decimal(str(fee.owner_settlement_amount)) for fee, _ in
rows), Decimal("0"))
    adjustment_sum = sum((Decimal(str(a.amount)) for a in adjustments),
Decimal("0"))
    total = fee_sum + adjustment_sum
    if total <= 0:
        raise BadRequestException("This café has no outstanding balance to pay
out.")

    payout = CafePayout(
        id=_uuid.uuid4(),
        caffe_id=caffe_id,
        amount=float(total),
        utr_reference=utr_reference,
        payment_method=payment_method,
        status=CafePayoutStatus.PAID,
        notes=notes,
        proof_image_url=proof_image_url,
        admin_note=admin_note,
        created_by_admin_id=admin_id,
        paid_at=paid_at or datetime.now(timezone.utc),
        destination_type=PayoutDestinationType(resolved_destination_type),
        destination_upi_vpa=payout_account.upi_vpa,

destination_bank_account_masked=payout_account.bank_account_number_masked,
        destination_bank_ifsc=payout_account.bank_ifsc,
        destination_account_holder_name=payout_account.account_holder_name,
        destination_payout_account_id=payout_account.id,
        destination_payout_account_version=payout_account.version,
    )
    self.db.add(payout)
    await self.db.flush()

```

The rest of the method (the `for fee, booking in rows:` loop through the final `return payout`) is unchanged.

- ☐ **Step 4: Run tests to verify they pass**

Run: `cd backend && pytest tests/test_cafe_payout_repository.py -v`

Expected: all PASS, including the five new tests and every pre-existing test in this file (the pre-existing tests never pass)

`destination_type/expected_payout_account_version`, exercising the inference/skip-check defaults).

- ☐ **Step 5: Wire the new fields through the admin API layer**

```
# backend/app/api/v1/admin_cafe_payouts.py – update the imports
from typing import Literal, Optional
from fastapi import APIRouter, Depends, Query, Response, status
from sqlalchemy.exc import IntegrityError

from app.api.deps import require_admin
from app.core.exceptions import BadRequestException, ConflictException,
NotFoundException
```

Replace `CafePayoutCreateRequest`:

```
class CafePayoutCreateRequest(BaseModel):
    utrReference: str
    paymentMethod: str
    destinationType: Literal["upi", "bank"]
    expectedPayoutAccountVersion: int
    confirmedPaymentMade: Literal[True]
    notes: Optional[str] = None
    proofImageUrl: Optional[str] = None
    adminNote: Optional[str] = None
    paidAt: Optional[datetime] = None
```

Replace the body of `create_cafe_payout` (the `repo.create_payout(...)` call):

```
payout = await repo.create_payout(
    cafe_id=cafe_id,
    admin_id=current_admin.id,
    utr_reference=payload.utrReference,
    payment_method=payload.paymentMethod,
    destination_type=payload.destinationType,
    expected_payout_account_version=payload.expectedPayoutAccountVersion,
    notes=payload.notes,
    proof_image_url=payload.proofImageUrl,
    admin_note=payload.adminNote,
    paid_at=payload.paidAt,
    audit_log_data={
        "admin_id": current_admin.id,
```

```

        "admin_email": current_admin.email,
        "action": "cafe_payout.create",
        "entity_type": "cafe_payout",
        "entity_name": cafe.name if cafe else None,
        "reason": payload.notes,
    },
)

```

The route's `except IntegrityError:` block is unchanged. `ConflictException` needs no special `except` clause here — it's a `BaseAppException` subclass already handled globally by `app/main.py`'s `custom_app_exception_handler`, which returns the correct 409 status and `error_code`.

- ❑ **Step 6: Update existing tests to supply the newly-required fields**

`backend/tests/test_cafe_payout_proof.py` — both POST payloads need the three new fields. Find:

```

"utrReference": "UTR-PROOF-1",
"paymentMethod": "upi",

```

and add immediately after:

```

"destinationType": "upi",
"expectedPayoutAccountVersion": 1,
"confirmedPaymentMade": True,

```

Find:

```

json={"utrReference": "UTR-NOPROOF-1", "paymentMethod": "neft"},

```

Replace with:

```

json={
    "utrReference": "UTR-NOPROOF-1", "paymentMethod": "neft",
    "destinationType": "upi", "expectedPayoutAccountVersion": 1,
    "confirmedPaymentMade": True,
},

```

`backend/tests/test_admin_cafe_payouts_api.py` — three call sites. Find:

```
    json={"utrReference": "UTR999", "paymentMethod": "neft", "notes": "test  
payout"},
```

Replace with:

```
    json={  
        "utrReference": "UTR999", "paymentMethod": "neft", "notes": "test  
payout",  
        "destinationType": "upi", "expectedPayoutAccountVersion": 1,  
        "confirmedPaymentMade": True,  
    },
```

Find:

```
    json={"utrReference": "UTR000", "paymentMethod": "neft"},
```

Replace with:

```
    json={  
        "utrReference": "UTR000", "paymentMethod": "neft",  
        "destinationType": "upi", "expectedPayoutAccountVersion": 1,  
        "confirmedPaymentMade": True,  
    },
```

Find:

```
    json={"utrReference": "UTR-RACE-API", "paymentMethod": "neft"},
```

Replace with:

```
    json={  
        "utrReference": "UTR-RACE-API", "paymentMethod": "neft",  
        "destinationType": "upi", "expectedPayoutAccountVersion": 1,  
        "confirmedPaymentMade": True,  
    },
```

- ☐ **Step 7: Run the full admin cafe-payouts test suite**

Run: `cd backend && pytest tests/test_admin_cafe_payouts_api.py tests/test_cafe_payout_proof.py tests/test_admin_cafe_payout_hold.py tests/test_cafe_payout_repository.py -v` Expected: all PASS.

- ❑ **Step 8: Commit**

```
git add backend/app/repositories/cafe_payout_repository.py
backend/app/api/v1/admin_cafe_payouts.py
backend/tests/test_cafe_payout_repository.py
backend/tests/test_cafe_payout_proof.py
backend/tests/test_admin_cafe_payouts_api.py
git commit -m "feat(payouts): snapshot destination + staleness check on
create_payout"
```

CafePayout now records which UPI/bank destination was actually paid to, frozen at creation time – a later owner edit to OwnerPayoutAccount can never rewrite a past payout's meaning. The admin-facing API requires an explicit `destinationType` and `expectedPayoutAccountVersion`; a version mismatch (owner changed details between the admin opening the payable and submitting) is rejected with 409 `PAYOUT_DESTINATION_STALE` before any write. The repository-level defaults (inferred type, skipped check when no version given) keep every pre-existing direct-repository test working unchanged."

Task 5: Payables list/breakdown serializer completeness + reveal endpoint

Files:

- Modify: `backend/app/repositories/cafe_payout_repository.py`
- Modify: `backend/app/api/v1/admin_cafe_payouts.py`
- Test: extend `backend/tests/test_admin_cafe_payouts_api.py`

Interfaces:

- Consumes: `payout_encryption.decrypt_bank_account_number` (existing), `AdminAuditLog` (existing).
- Produces: `CafePayoutRepository.list_cafes_with_outstanding()` rows now include `hasBank: bool`. New `CafePayoutRepository.get_payout_destination_summary(cafe_id) -> Optional[dict]` returning `{upiVpa, bankAccountNumberMasked, bankIfsc, accountHolderName, payoutAccountId, payoutAccountVersion,`

updatedAt}. GET /{cafe_id}/breakdown response gains a destination key. New POST /{cafe_id}/reveal-destination endpoint returning decrypted bank details, Cache-Control: no-store. Consumed by Task 9 (admin.ts) and Task 10 (modal).

- ❑ **Step 1: Write the failing tests**

Append to backend/tests/test_admin_cafe_payouts_api.py:

```
@pytest.mark.asyncio
async def test_outstanding_list_reports_has_bank_for_bank_only_cafe(db_session):
    """A café with only bank details (no UPI) must not be reported as
    lacking payout info – this was the concrete bug behind Priority 1."""
    admin = await _make_admin(db_session)
    gamer = await _make_gamer(db_session, "bankonly_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=95.0)
    db_session.add(fee)

    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe
    from app.core.payout_encryption import encrypt_bank_account_number
    from sqlalchemy import select as _select
    cafe_row = (await db_session.execute(_select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(
        id=uuid4(), owner_id=cafe_row.owner_id, upi_vpa=None,

bank_account_number_encrypted=encrypt_bank_account_number("9180200192847291"),
        bank_account_number_masked="••••7291", bank_ifsc="HDFC0000128",
        account_holder_name="Bank Only Owner",
    ))
    await db_session.commit()

    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(admin, is_admin=True)
        res = await client.get("/api/v1/admin/cafe-payouts/outstanding",
headers=headers)
        assert res.status_code == 200
        row = next(c for c in res.json()["data"]["cafes"] if c["cafeId"] ==
str(booking.cafe_id))
        assert row["upiVpa"] is None
        assert row["hasBank"] is True
        assert row["payoutDestinationSubmitted"] is True

@pytest.mark.asyncio
async def test_breakdown_includes_masked_destination_and_version(db_session):
    admin = await _make_admin(db_session)
    gamer = await _make_gamer(db_session, "breakdown_dest_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    fee = PlatformFee(id=uuid4(), booking_id=booking.id,
owner_settlement_amount=50.0)
```

```

db_session.add(fee)

from app.models.owner_payout_account import OwnerPayoutAccount
from app.models.cafe import Cafe
from sqlalchemy import select as _select
cafe_row = (await db_session.execute(_select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
db_session.add(OwnerPayoutAccount(
    id=uuid4(), owner_id=cafe_row.owner_id, upi_vpa="breakdown@okaxis",
    account_holder_name="Breakdown Holder", version=1,
))
await db_session.commit()

async with AsyncClient(app=app, base_url="http://test") as client:
    headers = auth_headers(admin, is_admin=True)
    res = await client.get(f"/api/v1/admin/cafe-
payouts/{booking.cafe_id}/breakdown", headers=headers)
    assert res.status_code == 200
    destination = res.json()["data"]["destination"]
    assert destination["upiVpa"] == "breakdown@okaxis"
    assert destination["accountHolderName"] == "Breakdown Holder"
    assert destination["payoutAccountVersion"] == 1

@pytest.mark.asyncio
async def
test_reveal_destination_returns_decrypted_bank_number_and_logs_audit(db_session):
    admin = await _make_admin(db_session)
    gamer = await _make_gamer(db_session, "reveal_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)

    from app.models.owner_payout_account import OwnerPayoutAccount
    from app.models.cafe import Cafe
    from app.core.payout_encryption import encrypt_bank_account_number
    from sqlalchemy import select as _select
    cafe_row = (await db_session.execute(_select(Cafe).where(Cafe.id ==
booking.cafe_id))).scalars().first()
    db_session.add(OwnerPayoutAccount(
        id=uuid4(), owner_id=cafe_row.owner_id,

bank_account_number_encrypted=encrypt_bank_account_number("9180200192847291"),
    bank_account_number_masked="....7291", bank_ifsc="HDFC0000128",
    account_holder_name="Reveal Holder", version=1,
    ))
    await db_session.commit()

    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(admin, is_admin=True)
        res = await client.post(
            f"/api/v1/admin/cafe-payouts/{booking.cafe_id}/reveal-destination",
headers=headers
        )
        assert res.status_code == 200
        assert res.json()["data"]["bankAccountNumber"] == "9180200192847291"
        assert res.headers["cache-control"] == "no-store"

        audit_res = await client.get(
            "/api/v1/admin/audit-log?entityType=owner_payout_account",
headers=headers

```

```

)
assert audit_res.status_code == 200
items = audit_res.json()["data"]["items"]
assert any(a["action"] == "payout_destination.revealed" for a in items)
# The decrypted number must never appear in the audit log itself.
assert not any("9180200192847291" in str(a) for a in items)

```

- ❑ **Step 2: Run tests to verify they fail**

Run: `cd backend && pytest tests/test_admin_cafe_payouts_api.py -v -k "has_bank or breakdown_includes or reveal_destination"` Expected: FAIL — `hasBank` key absent, `destination` key absent, `/reveal-destination` route doesn't exist (404).

- ❑ **Step 3: Add `hasBank` to `list_cafes_with_outstanding`**

```

# backend/app/repositories/cafe_payout_repository.py – replace the body of
list_cafes_with_outstanding's account/dict block
    async def list_cafes_with_outstanding(self) -> list[dict]:
        cafes_result = await self.db.execute(select(Cafe.id, Cafe.name,
        Cafe.owner_id, Cafe.payout_on_hold, Cafe.payout_hold_reason))
        out = []
        for cafe_id, cafe_name, owner_id, on_hold, hold_reason in
cafes_result.all():
            amount = await self.get_outstanding_amount(cafe_id)
            if amount > 0:
                account = (await self.db.execute(
                    select(OwnerPayoutAccount).where(OwnerPayoutAccount.owner_id ==
owner_id)
                )).scalars().first()
                has_bank = bool(
                    account
                    and account.bank_account_number_encrypted
                    and account.bank_ifsc
                    and account.account_holder_name
                )
                destination_submitted = bool(account and (account.upi_vpa or
has_bank))
                out.append({
                    "cafeId": str(cafe_id),
                    "cafeName": cafe_name,
                    "outstandingAmount": float(amount),
                    "payoutDestinationSubmitted": destination_submitted,
                    "upiVpa": account.upi_vpa if account else None,
                    "hasBank": has_bank,
                    "payoutOnHold": on_hold,
                    "payoutHoldReason": hold_reason,
                })
            return out

```

- ❑ **Step 4: Add `get_payout_destination_summary`**

Add this new method to `CafePayoutRepository`, right after

`get_outstanding_breakdown`:

```
    async def get_payout_destination_summary(self, cafe_id: UUID) ->
Optional[dict]:
    cafe_row = (await self.db.execute(select(Cafe.owner_id).where(Cafe.id ==
cafe_id))).first()
    if not cafe_row:
        return None
    account = (await self.db.execute(
        select(OwnerPayoutAccount).where(OwnerPayoutAccount.owner_id ==
cafe_row.owner_id)
    )).scalars().first()
    if not account:
        return None
    return {
        "upiVpa": account.upi_vpa,
        "bankAccountNumberMasked": account.bank_account_number_masked,
        "bankIfsc": account.bank_ifsc,
        "accountHolderName": account.account_holder_name,
        "payoutAccountId": str(account.id),
        "payoutAccountVersion": account.version,
        "updatedAt": account.updated_at.isoformat(),
    }
```

- ☐ **Step 5: Wire the destination summary into the breakdown endpoint, add the reveal endpoint**

```
# backend/app/api/v1/admin_cafe_payouts.py – replace get_cafe_payout_breakdown
@router.get("/{cafe_id}/breakdown", status_code=status.HTTP_200_OK)
async def get_cafe_payout_breakdown(
    cafe_id: UUID,
    current_admin: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    repo = CafePayoutRepository(db)
    bookings = await repo.get_outstanding_breakdown(cafe_id)
    destination = await repo.get_payout_destination_summary(cafe_id)
    return {"success": True, "data": {"bookings": bookings, "destination":
destination}}
```

Add the reveal endpoint, right after `get_cafe_payout_breakdown`:

```
@router.post("/{cafe_id}/reveal-destination", status_code=status.HTTP_200_OK)
async def reveal_cafe_payout_destination(
    cafe_id: UUID,
    response: Response,
    current_admin: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
```

```

):
    """Decrypts the owner's bank account number just for this response, for
    the admin to actually type into their banking app during a manual
    transfer. Masked display everywhere else is unchanged – this is the
    one explicit, audited, non-cacheable exception. The decrypted value
    must never be logged."""
    cafe = await CafeRepository(db).get_by_id(cafe_id)
    if not cafe:
        raise NotFoundException("Café not found")

    account = (await db.execute(
        select(OwnerPayoutAccount).where(OwnerPayoutAccount.owner_id ==
cafe.owner_id)
    )).scalars().first()
    if not account:
        raise NotFoundException("This café has no payout details on file")

    from app.core.payout_encryption import decrypt_bank_account_number
    bank_account_number = None
    if account.bank_account_number_encrypted:
        try:
            bank_account_number =
decrypt_bank_account_number(account.bank_account_number_encrypted)
        except Exception:
            bank_account_number = None

    db.add(AdminAuditLog(
        id=_uuid.uuid4(),
        admin_id=current_admin.id,
        admin_email=current_admin.email,
        action="payout_destination.revealed",
        entity_type="owner_payout_account",
        entity_id=str(account.id),
        entity_name=cafe.name,
        reason=None,
    ))
    await db.commit()

    response.headers["Cache-Control"] = "no-store"
    return {
        "success": True,
        "data": {
            "upiVpa": account.upi_vpa,
            "bankAccountNumber": bank_account_number,
            "bankIfsc": account.bank_ifsc,
            "accountHolderName": account.account_holder_name,
            "payoutAccountId": str(account.id),
            "payoutAccountVersion": account.version,
        },
    }
}

```

Add the missing import at the top of the file (`OwnerPayoutAccount`, `select` already imported via `sqlalchemy` — confirm `select` is imported; if not add it):

```
from sqlalchemy import select
from app.models.owner_payout_account import OwnerPayoutAccount
```

- ☐ **Step 6: Run tests to verify they pass**

Run: `cd backend && pytest tests/test_admin_cafe_payouts_api.py -v`

Expected: all PASS.

- ☐ **Step 7: Commit**

```
git add backend/app/repositories/cafe_payout_repository.py
backend/app/api/v1/admin_cafe_payouts.py
backend/tests/test_admin_cafe_payouts_api.py
git commit -m "feat(payouts): expose bank fields in payables list/breakdown, add
reveal endpoint"
```

`list_cafes_with_outstanding` now reports `hasBank` alongside `upiVpa`, so a bank-only café no longer shows a false 'no UPI on file' warning. The `breakdown` endpoint now returns the owner's current masked destination + version. A new POST `/{{cafe_id}}/reveal-destination` endpoint decrypts the bank account number just-in-time for an admin actually making a manual transfer – re-authorized independently, non-cacheable, and audit-logged (action only, never the decrypted value)."

Task 6:

NotificationService.send_payout_details_changed

Files:

- Modify: `backend/app/services/notification_service.py`

Interfaces:

- Produces: `NotificationService.send_payout_details_changed(self, email: str, full_name: str) -> bool`. Consumed by Task 7.

- ☐ **Step 1: Add the method**

Add this method to `NotificationService`, right after `send_password_reset` (around line 275):

```

    async def send_payout_details_changed(self, email: str, full_name: str) ->
bool:
    try:
        subject = "Your KHEL-O payout details were changed"
        html_body = _email_wrapper(f"""
            <h2 style="margin-top: 0; color: {_BRAND_TEXT_PRIMARY};">Payout
details updated</h2>
            <p>Hello <strong>{full_name or 'there'}</strong>,</p>
            <p>The payout destination on your KHEL-O café account was just
changed. This is where
            your future weekly payouts will be sent.</p>
            <p style="color: {_BRAND_TEXT_SECONDARY};">If you made this change,
no action is needed.
            If you didn't, please contact KHEL-O support immediately and change
your account
            password.</p>
            """)
        return await self._send_resend_email(email, subject, html_body,
"PAYOUT-DETAILS-CHANGED")
    except Exception as e:
        logger.error("send_payout_details_changed_error", error=str(e),
email=email)
        return False

```

- ☐ **Step 2: Verify it imports and runs cleanly**

Run: `cd backend && python -c "from app.services.notification_service import NotificationService; assert hasattr(NotificationService, 'send_payout_details_changed'); print('OK')"` Expected: prints OK.

- ☐ **Step 3: Commit**

```

git add backend/app/services/notification_service.py
git commit -m "feat(notifications): add send_payout_details_changed email

```

Informational, fired after a successful owner payout-details change so the owner notices a change they didn't make."

Task 7: Owner payout-details endpoints (replaces legacy `/setup /status`)

Files:

- Modify: `backend/app/api/v1/owner_payouts.py`
- Modify: `backend/app/repositories/cafe_payout_repository.py`

- Create: `backend/app/schemas/owner_payout_destination.py`
- Test: Create `backend/tests/test_owner_payout_destination_api.py`

Interfaces:

- Consumes: `OwnerPayoutRepository.upsert_payout_details` (existing, Task 3's version bump), `NotificationService.send_payout_details_changed` (Task 6), `OwnerAuditLog` (Task 2), `verify_password` from `app.core.security`.
- Produces: `GET /api/v1/owner/payouts/destination`, `PATCH /api/v1/owner/payouts/destination`.
`CafePayoutRepository.get_outstanding_amount_for_owner_cafes(caffe_ids: list[UUID]) -> Decimal`. Consumed by Task 8 (frontend).
- The legacy `GET /status` and `POST /setup` routes are removed — their only consumer (`PayoutSetupCard.tsx`) is deleted in Task 8. `OwnerPayoutService`, `handle_kyc_webhook`, and `kyc_status/razorpay_account_id` are **not** touched (live consumers: `payment_service.py:659`, `admin_service.py:733` — confirmed via repo-wide search; per the Global Constraints, only the two now-orphaned routes are removed).
- ☐ **Step 1: Confirm the dependency audit findings (no code change — documentation of what was checked)**

This step records the audit already performed for this plan, so the removal in Step 3 is traceable rather than an assumption:

- `owner_payouts` grep → only mount point is `router.py:11,32`; the only routes with no other consumer are `/status` and `/setup`.
- `PayoutSetupCard/setupPayout/getPayoutStatus` grep → exactly 4 frontend files, all either defining or consuming these two functions/component (`lib/api/owner.ts`, `PayoutSetupCard.tsx`, `owner/settings/page.tsx`, and `lib/api.ts` which is an unrelated barrel re-export) — no other consumer exists.
- `handle_kyc_webhook` → live consumer at `backend/app/services/payment_service.py:659` (Razorpay webhook handler). **Not removed.**
- `kyc_status` → live consumer at `backend/app/services/admin_service.py:733` (admin verification-queue query). **Not removed.**

- Conclusion: safe to delete the `/status` and `/setup` **routes** and their now-unused `PayoutAccountCreateRequest/OwnerPayoutService` **usages in this router file only** — `OwnerPayoutService` the class, and the model fields it manages, stay.
- ☐ **Step 2: Write the schema**

```
# backend/app/schemas/owner_payout_destination.py
from typing import Optional
from pydantic import BaseModel, ConfigDict

def to_camel(string: str) -> str:
    components = string.split('_')
    return components[0] + ''.join(x.title() for x in components[1:])

class OwnerPayoutDestinationUpdateRequest(BaseModel):
    current_password: str
    upi_vpa: Optional[str] = None
    bank_account_number: Optional[str] = None
    bank_ifsc: Optional[str] = None
    account_holder_name: Optional[str] = None
    bank_name: Optional[str] = None
    account_type: Optional[str] = None
    business_pan: Optional[str] = None

    model_config = ConfigDict(alias_generator=to_camel, populate_by_name=True)
```

- ☐ **Step 3: Write the failing API tests**

```
# backend/tests/test_owner_payout_destination_api.py
import uuid
import pytest
from httpx import AsyncClient
from sqlalchemy import select

from app.main import app
from app.models.user import User, UserRole
from app.models.user_role import UserRoleMapping
from app.models.cafe import Cafe, VerificationStatus
from app.models.owner_payout_account import OwnerPayoutAccount
from app.models.owner_audit_log import OwnerAuditLog
from app.core.security import get_password_hash
from tests.conftest import auth_headers

async def _make_owner_with_cafe(db_session, password="testpass123"):
    owner = User(
        id=uuid.uuid4(),
        email=f"payout_dest_owner_{uuid.uuid4().hex[:8]}@test.com",
        full_name="Payout Dest Owner", password_hash=get_password_hash(password),
        role=UserRole.CAFE_OWNER, is_active=True,
```

```

    )
    db_session.add(owner)
    await db_session.flush()
    db_session.add(UserRoleMapping(id=uuid.uuid4(), user_id=owner.id,
role=UserRole.CAFE_OWNER))
    cafe = Cafe(
        id=uuid.uuid4(), owner_id=owner.id, name="Payout Dest Café",
address_line1="1 Test St",
        city="Bengaluru", state="Karnataka", pincode="560001",
phone_number="+919876543210",
        verification_status=VerificationStatus.VERIFIED, is_active=True,
        opening_time=None, closing_time=None, bookable_stations=10,
    )
    db_session.add(cafe)
    await db_session.commit()
    return owner, cafe

```

@pytest.mark.asyncio

```

async def test_get_destination_returns_none_when_not_set(db_session):
    owner, _cafe = await _make_owner_with_cafe(db_session)
    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(owner, is_admin=False)
        res = await client.get("/api/v1/owner/payouts/destination",
headers=headers)
        assert res.status_code == 200
        assert res.json()["data"]["destination"] is None

```

@pytest.mark.asyncio

```

async def test_patch_destination_rejects_wrong_password(db_session):
    owner, _cafe = await _make_owner_with_cafe(db_session,
password="correctpass123")
    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(owner, is_admin=False)
        res = await client.patch(
            "/api/v1/owner/payouts/destination",
            json={"currentPassword": "wrongpass", "upiVpa": "new@okaxis"},
            headers=headers,
        )
        assert res.status_code == 401

```

```

account = (await db_session.execute(
    select(OwnerPayoutAccount).where(OwnerPayoutAccount.owner_id == owner.id)
)).scalars().first()
assert account is None # nothing was written

```

```

audit_rows = (await db_session.execute(
    select(OwnerAuditLog).where(OwnerAuditLog.owner_id == owner.id)
)).scalars().all()
assert audit_rows == []

```

@pytest.mark.asyncio

```

async def test_patch_destination_succeeds_and_writes_masked_audit_log(db_session):
    owner, _cafe = await _make_owner_with_cafe(db_session,
password="correctpass123")
    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(owner, is_admin=False)

```

```

        res = await client.patch(
            "/api/v1/owner/payouts/destination",
            json={
                "currentPassword": "correctpass123",
                "upiVpa": "newdest@okaxis",
                "accountHolderName": "New Holder",
            },
            headers=headers,
        )
        assert res.status_code == 200, res.text
        assert res.json()["data"]["destination"]["upiVpa"] == "newdest@okaxis"
        assert res.json()["data"]["destination"]["version"] == 1

    audit_rows = (await db_session.execute(
        select(OwnerAuditLog).where(OwnerAuditLog.owner_id == owner.id)
    )).scalars().all()
    assert len(audit_rows) == 1
    assert audit_rows[0].action == "payout_details.updated"
    assert "newdest@okaxis" in audit_rows[0].after_summary
    # Bank account number, if any, must never appear in plaintext.
    assert "password" not in audit_rows[0].after_summary.lower()

@pytest.mark.asyncio
async def
test_patch_destination_blocks_clearing_only_destination_with_outstanding_balance(db
_session):
    from tests.test_admin_v2_features import _make_gamer,
_make_booking_with_payment
    from app.models.platform_fee import PlatformFee

    owner, cafe = await _make_owner_with_cafe(db_session,
password="correctpass123")
    db_session.add(OwnerPayoutAccount(id=uuid.uuid4(), owner_id=owner.id,
upi_vpa="onlydest@okaxis"))
    await db_session.commit()

    gamer = await _make_gamer(db_session, "block_clear_gamer")
    booking, payment = await _make_booking_with_payment(db_session, gamer)
    # Reassign the booking to this owner's café so the outstanding balance
    # check has something real to find.
    booking.cafe_id = cafe.id
    db_session.add(PlatformFee(booking_id=booking.id,
owner_settlement_amount=100.0))
    await db_session.commit()

    async with AsyncClient(app=app, base_url="http://test") as client:
        headers = auth_headers(owner, is_admin=False)
        res = await client.patch(
            "/api/v1/owner/payouts/destination",
            json={"currentPassword": "correctpass123", "upiVpa": None,
"bankAccountNumber": None},
            headers=headers,
        )
        assert res.status_code == 400
        assert "outstanding" in res.json()["error"]["message"].lower()

```

- ☐ **Step 4: Run tests to verify they fail**

Run: `cd backend && pytest tests/test_owner_payout_destination_api.py -v`
Expected: FAIL — `/api/v1/owner/payouts/destination` doesn't exist yet (404 on every request).

- ☐ **Step 5: Add `get_outstanding_amount_for_owner_cafes` to `CafePayoutRepository`**

Add this method right after `get_outstanding_amount`:

```
async def get_outstanding_amount_for_owner_cafes(self, cafe_ids: list[UUID]) -> Decimal:
    total = Decimal("0")
    for cafe_id in cafe_ids:
        total += await self.get_outstanding_amount(cafe_id)
    return total
```

- ☐ **Step 6: Replace the legacy routes with the new destination endpoints**

```
# backend/app/api/v1/owner_payouts.py – full replacement
from decimal import Decimal

from fastapi import APIRouter, Depends, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_db
from app.api.deps import require_cafe_owner
from app.core.exceptions import AuthException, BadRequestException
from app.core.security import verify_password
from app.models.cafe import Cafe
from app.models.owner_audit_log import OwnerAuditLog
from app.models.user import User
from app.repositories.cafe_payout_repository import CafePayoutRepository
from app.repositories.owner_payout_repository import OwnerPayoutRepository
from app.schemas.owner_payout_destination import OwnerPayoutDestinationUpdateRequest
from app.services.notification_service import NotificationService

router = APIRouter()

def _mask_summary(account) -> str:
    if not account:
        return "No destination on file"
    parts = []
    if account.upi_vpa:
        parts.append(f"UPI: {account.upi_vpa}")
    if account.bank_account_number_masked:
        parts.append(f"Bank: {account.bank_account_number_masked} / IFSC
```

```
{account.bank_ifsc}")
    return " | ".join(parts) if parts else "No destination on file"
```

```
def _destination_response(account) -> dict | None:
    if not account:
        return None
    return {
        "upiVpa": account.upi_vpa,
        "bankAccountNumberMasked": account.bank_account_number_masked,
        "bankIfsc": account.bank_ifsc,
        "accountHolderName": account.account_holder_name,
        "version": account.version,
        "updatedAt": account.updated_at.isoformat(),
    }
```

```
@router.get("/cafe-payouts", status_code=status.HTTP_200_OK)
```

```
async def get_owner_cafe_payouts(
    current_owner: User = Depends(require_cafe_owner),
    db: AsyncSession = Depends(get_db),
):
    # An owner can have multiple cafés. This must aggregate across ALL of
    # them (outstanding balance summed, history merged) to agree with
    # /payouts/summary on the same owner payouts screen, which already
    # aggregates across every café the owner owns – picking just the
    # most-recently-created café here would silently hide any other café's
    # balance/history.
    cafe_stmt = (
        select(Cafe)
        .where(Cafe.owner_id == current_owner.id)
        .order_by(Cafe.created_at.desc())
    )
    cafes = (await db.execute(cafe_stmt)).scalars().all()

    if not cafes:
        return {"success": True, "data": {"outstandingAmount": 0.0, "history": []}}

    cafe_ids = [c.id for c in cafes]
    repo = CafePayoutRepository(db)

    outstanding = Decimal("0")
    for cafe_id in cafe_ids:
        outstanding += await repo.get_outstanding_amount(cafe_id)

    history = await repo.list_payouts(cafe_id=cafe_ids)

    return {
        "success": True,
        "data": {
            "outstandingAmount": float(outstanding),
            "history": history["items"],
        },
    }
```

```
@router.get("/destination", status_code=status.HTTP_200_OK)
```

```
async def get_payout_destination(
    current_owner: User = Depends(require_cafe_owner),
```

```

db: AsyncSession = Depends(get_db),
):
    account = await OwnerPayoutRepository(db).get_by_owner_id(current_owner.id)
    return {"success": True, "data": {"destination":
_destination_response(account)}}

@router.patch("/destination", status_code=status.HTTP_200_OK)
async def update_payout_destination(
    payload: OwnerPayoutDestinationUpdateRequest,
    current_owner: User = Depends(require_cafe_owner),
    db: AsyncSession = Depends(get_db),
):
    if not verify_password(payload.current_password, current_owner.password_hash):
        raise AuthException("Incorrect password.")

    payout_repo = OwnerPayoutRepository(db)
    existing = await payout_repo.get_by_owner_id(current_owner.id)
    before_summary = _mask_summary(existing)

    would_clear_destination = not payload.upi_vpa and not
payload.bank_account_number
    if would_clear_destination:
        cafe_ids = [
            row[0] for row in
                (await db.execute(select(Cafe.id).where(Cafe.owner_id ==
current_owner.id))).all()
        ]
        outstanding = await
CafePayoutRepository(db).get_outstanding_amount_for_owner_cafes(cafe_ids)
        if outstanding > 0:
            raise BadRequestException(
                "You can't remove your only payout destination while you have an
outstanding "
                "balance – add a valid UPI ID or bank account first, or wait until
it's paid out."
            )

    account = await payout_repo.upsert_payout_details(
        owner_id=current_owner.id,
        upi_vpa=payload.upi_vpa,
        bank_account_number=payload.bank_account_number,
        bank_ifsc=payload.bank_ifsc,
        account_holder_name=payload.account_holder_name,
        bank_name=payload.bank_name,
        account_type=payload.account_type,
        business_pan=payload.business_pan,
        default_holder_name=current_owner.full_name,
    )
    after_summary = _mask_summary(account)

    db.add(OwnerAuditLog(
        owner_id=current_owner.id,
        action="payout_details.updated",
        entity_type="owner_payout_account",
        entity_id=str(account.id),
        before_summary=before_summary,
        after_summary=after_summary,
    ))

```

```

await db.commit()
await db.refresh(account)

await NotificationService().send_payout_details_changed(
    email=current_owner.email, full_name=current_owner.full_name,
)

return {"success": True, "data": {"destination":
    _destination_response(account)}}

```

- ☐ **Step 7: Run tests to verify they pass**

Run: `cd backend && pytest tests/test_owner_payout_destination_api.py -v` Expected: all PASS.

- ☐ **Step 8: Run the broader owner-payouts and webhook-adjacent suites to confirm nothing else broke**

Run: `cd backend && pytest tests/test_owner_cafe_payouts_api.py tests/test_owner_payout_account_model.py -v` Expected: all PASS — `/cafe-payouts` is unchanged in this file, only `/status` and `/setup` were removed (neither is exercised by these two files, confirmed by their names/scope).

- ☐ **Step 9: Commit**

```

git add backend/app/api/v1/owner_payouts.py
backend/app/repositories/cafe_payout_repository.py
backend/app/schemas/owner_payout_destination.py
backend/tests/test_owner_payout_destination_api.py
git commit -m "feat(payouts): add owner payout-details GET/PATCH, retire legacy
/setup /status

```

Owners can now view and edit their UPI/bank payout destination after onboarding – previously there was no edit path at all. Changing it requires the current account password; a wrong password makes no change, sends no email, writes no audit row. Clearing the only destination while an outstanding balance exists is rejected. Every change fires a confirmation email and writes a masked-only OwnerAuditLog entry. The legacy `/setup` and `/status` routes (whose only consumer was the frontend PayoutSetupCard, removed in the next commit) are deleted; OwnerPayoutService, handle_kyc_webhook, and kyc_status are left untouched – confirmed live consumers exist in payment_service.py's webhook handler and admin_service.py's verification queue."

Task 8: Frontend — owner settings payout editor

Files:

- Delete: `frontend/src/components/owner/PayoutSetupCard.tsx`
- Create: `frontend/src/components/owner/PayoutDetailsCard.tsx`
- Modify: `frontend/src/lib/api/owner.ts:1-10,168-174`
- Modify: `frontend/src/app/(owner)/owner/settings/page.tsx:12,179`

Interfaces:

- Consumes: `GET /api/v1/owner/payouts/destination`, `PATCH /api/v1/owner/payouts/destination` (Task 7).
- Produces: `getPayoutDestination(): Promise<{ destination: PayoutDestination | null }>`, `updatePayoutDestination(body: PayoutDestinationUpdateRequest): Promise<{ destination: PayoutDestination }>` in `lib/api/owner.ts`. `<PayoutDetailsCard />` component.
- ☐ **Step 1: Replace the two legacy functions in `lib/api/owner.ts`**

Remove the `getPayoutStatus/setupPayout` functions (lines 168-174) and replace with:

```
export interface PayoutDestination {
  upiVpa: string | null;
  bankAccountNumberMasked: string | null;
  bankIfsc: string | null;
  accountHolderName: string | null;
  version: number;
  updatedAt: string;
}

export interface PayoutDestinationUpdateRequest {
  currentPassword: string;
  upiVpa?: string | null;
  bankAccountNumber?: string | null;
  bankIfsc?: string | null;
  accountHolderName?: string | null;
  bankName?: string | null;
  accountType?: string | null;
  businessPan?: string | null;
}

export async function getPayoutDestination(): Promise<{ destination:
PayoutDestination | null }> {
  return call(() => apiClient.get('/api/v1/owner/payouts/destination'));
}

export async function updatePayoutDestination(
```

```

body: PayoutDestinationUpdateRequest,
): Promise<{ destination: PayoutDestination }> {
  return call(( ) => apiClient.patch('/api/v1/owner/payouts/destination', body));
}

```

Remove the now-unused **OwnerPayoutAccount**, **PayoutSetupRequest** names from the top-of-file type import (line 6-10) — leave **OwnerDashboard**, **OwnerBookingListResponse**, **OwnerBookingParams**, **BookingDetail**, **User** as they still have consumers elsewhere in this file:

```

import type {
  OwnerDashboard,
  OwnerBookingListResponse,
  OwnerBookingParams,
  BookingDetail,
  User,
} from '@types';

```

- ☐ **Step 2: Delete the legacy component**

```
rm frontend/src/components/owner/PayoutSetupCard.tsx
```

- ☐ **Step 3: Create the new **PayoutDetailsCard****

```

// frontend/src/components/owner/PayoutDetailsCard.tsx
'use client';

import { useState, useEffect, type FormEvent } from 'react';
import { Landmark, AlertCircle, Pencil } from 'lucide-react';
import { Card, CardContent, Button, Input, Badge } from '@components/ui';
import { getPayoutDestination, updatePayoutDestination } from '@lib/api/owner';
import type { PayoutDestination } from '@lib/api/owner';

export function PayoutDetailsCard() {
  const [destination, setDestination] = useState<PayoutDestination | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [isEditing, setIsEditing] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [currentPassword, setCurrentPassword] = useState('');
  const [upiVpa, setUpiVpa] = useState('');
  const [bankAccountNumber, setBankAccountNumber] = useState('');
  const [bankIfsc, setBankIfsc] = useState('');
  const [accountHolderName, setAccountHolderName] = useState('');

  function loadDestination() {
    setIsLoading(true);
  }

```

```

    getPayoutDestination()
      .then((res) => setDestination(res.destination))
      .catch(() => setDestination(null))
      .finally(() => setIsLoading(false));
  }

  useEffect(() => {
    loadDestination();
  }, []);

  function startEdit() {
    setError(null);
    setCurrentPassword('');
    setUpiVpa(destination?.upiVpa ?? '');
    setBankAccountNumber('');
    setBankIfsc(destination?.bankIfsc ?? '');
    setAccountHolderName(destination?.accountHolderName ?? '');
    setIsEditing(true);
  }

  async function handleSubmit(e: FormEvent) {
    e.preventDefault();
    setError(null);
    setIsSubmitting(true);
    try {
      const res = await updatePayoutDestination({
        currentPassword,
        upiVpa: upiVpa || null,
        bankAccountNumber: bankAccountNumber || undefined,
        bankIfsc: bankIfsc || undefined,
        accountHolderName: accountHolderName || undefined,
      });
      setDestination(res.destination);
      setIsEditing(false);
    } catch (err: any) {
      setError(err?.message || 'Failed to update payout details.');
```

} finally {
 setIsSubmitting(false);
 }
 }

 if (isLoading) {
 return (
 <Card elevation="resting" className="bg-surface border border-border">
 <CardContent className="p-6">
 <div className="animate-pulse h-16 rounded-xl bg-surface-hover" />
 </CardContent>
 </Card>
);
 }

 return (
 <Card elevation="resting" className="bg-surface border border-border">
 <CardContent className="p-4 sm:p-6 flex flex-col gap-4">
 <div className="flex items-start gap-3.5">
 <div className="h-11 w-11 sm:h-12 sm:w-12 rounded-2xl flex items-center justify-center shrink-0 bg-emerald-500/10 text-emerald-600 border border-emerald-500/20">
 <Landmark className="h-6 w-6" />
 </div>
 </div>
 </CardContent>
 </Card>
);
}

```

    </div>
    <div className="flex flex-col gap-1.5 flex-1">
      <div className="flex items-center gap-2 flex-wrap">
        <h3 className="font-heading text-body font-bold text-text-
primary">Payout Details</h3>
        {destination && <Badge variant="default" size="sm">On file</Badge>}
      </div>
      <p className="text-caption text-text-secondary max-w-md">
        The UPI ID or bank account KHEL-0 sends your weekly payout to.
      </p>
    </div>
  </div>

  {!isEditing && (
    <>
      {destination ? (
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3 p-4 rounded-2xl
bg-surface-hover border border-border/80 text-caption">
          <div>
            <span className="text-xs text-text-tertiary block">UPI ID</span>
            <span className="font-bold text-text-primary">{destination.upiVpa
|| '-'}</span>
          </div>
          <div>
            <span className="text-xs text-text-tertiary block">Bank
Account</span>
            <span className="font-bold text-text-primary">
{destination.bankAccountNumberMasked || '-'}</span>
          </div>
          <div>
            <span className="text-xs text-text-tertiary block">Bank
IFSC</span>
            <span className="font-bold text-text-primary">
{destination.bankIfsc || '-'}</span>
          </div>
          <div>
            <span className="text-xs text-text-tertiary block">Last
Updated</span>
            <span className="font-bold text-text-primary">
{new Date(destination.updatedAt).toLocaleDateString()}
          </span>
          </div>
        </div>
      ) : (
        <p className="text-caption text-text-secondary">No payout details on
file yet.</p>
      )}
      <Button variant="secondary" onClick={startEdit} className="gap-2 self-
start">
        <Pencil className="h-4 w-4" />
        {destination ? 'Edit payout details' : 'Add payout details'}
      </Button>
    </>
  )}

  {isEditing && (
    <form onSubmit={handleSubmit} className="flex flex-col gap-3">
      {error && (
        <div className="flex items-center gap-2 p-3 rounded-xl bg-error/10

```

```

border border-error/20 text-caption text-error">
    <AlertCircle className="h-4 w-4 flex-shrink-0" />
    <span>{error}</span>
</div>
)}
<div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
    <Input
        label="UPI ID"
        placeholder="yourname@okhdfcbank"
        value={upiVpa}
        onChange={(e) => setUpiVpa(e.target.value)}
    />
    <Input
        label="Account Holder Name"
        placeholder="As per bank records"
        value={accountHolderName}
        onChange={(e) => setAccountHolderName(e.target.value)}
    />
    <Input
        label="Bank Account Number"
        placeholder="Leave blank to keep unchanged"
        value={bankAccountNumber}
        onChange={(e) => setBankAccountNumber(e.target.value.replace(/\D/g,
''))}
    />
    <Input
        label="Bank IFSC Code"
        placeholder="HDFC0000128"
        value={bankIfsc}
        onChange={(e) => setBankIfsc(e.target.value.toUpperCase())}
        maxLength={11}
    />
    <Input
        label="Current Password *"
        type="password"
        placeholder="Confirm it's you"
        value={currentPassword}
        onChange={(e) => setCurrentPassword(e.target.value)}
        required
        className="sm:col-span-2"
    />
</div>
<div className="flex items-center gap-2 self-end">
    <Button type="button" variant="ghost" onClick={() =>
setIsEditing(false)}>
        Cancel
    </Button>
    <Button type="submit" variant="primary" isLoading={isSubmitting}
loadingText="Saving...">
        Save payout details
    </Button>
</div>
</form>
)}
</CardContent>
</Card>
);
}

```

- ☐ **Step 4: Wire it into the settings page**

```
// frontend/src/app/(owner)/owner/settings/page.tsx — line 12
import { PayoutDetailsCard } from '@components/owner/PayoutDetailsCard';
```

```
// frontend/src/app/(owner)/owner/settings/page.tsx — line 179
<PayoutDetailsCard />
```

- ☐ **Step 5: Type-check and build**

Run: `cd frontend && npx tsc --noEmit -p tsconfig.json` Expected: no errors — in particular, no leftover reference to `PayoutSetupCard`, `getPayoutStatus`, or `setupPayout` anywhere in the frontend.

- ☐ **Step 6: Commit**

```
git add frontend/src/components/owner/PayoutDetailsCard.tsx
frontend/src/lib/api/owner.ts "frontend/src/app/(owner)/owner/settings/page.tsx"
git rm frontend/src/components/owner/PayoutSetupCard.tsx
git commit -m "feat(owner): replace legacy KYC payout card with a real payout-
details editor"
```

Owners can now edit their UPI/bank payout destination after onboarding (previously impossible) via a password-confirmed form against the new backend endpoints. The old 'Direct Bank Payouts' card — a disconnected Razorpay-Route/KYC leftover that never touched UPI and silently clobbered fields in the same account row — is removed."

Task 9: Frontend — admin `lib/api/admin.ts` extensions + error code

Files:

- Modify: `frontend/src/lib/api/admin.ts:255-339`
- Modify: `frontend/src/lib/api/errors.ts`

Interfaces:

- Consumes: GET /outstanding with hasBank (Task 5), GET /{cafeId}/breakdown with destination (Task 5), POST /{cafeId}/reveal-destination (Task 5), POST /{cafeId} with destinationType/expectedPayoutAccountVersion/confirmedPaymentMade (Task 4).
- Produces: AdminOutstandingCafePayout.hasBank, CafePayoutDestinationSummary type, getCafePayoutBreakdown returning { bookings, destination }, revealCafePayoutDestination(cafeId), extended createCafePayout body type, API_ERROR_CODES.PAYOUT_DESTINATION_STALE. Consumed by Task 10.
- ☐ **Step 1: Add the error code**

```
// frontend/src/lib/api/errors.ts – add to API_ERROR_CODES, after UNAUTHORIZED
PAYOUT_DESTINATION_STALE: 'PAYOUT_DESTINATION_STALE',
```

- ☐ **Step 2: Extend the admin.ts types and functions**

Replace AdminOutstandingCafePayout:

```
export interface AdminOutstandingCafePayout {
  cafeId: string;
  cafeName: string;
  outstandingAmount: number;
  payoutDestinationSubmitted: boolean;
  upiVpa: string | null;
  hasBank: boolean;
  payoutOnHold: boolean;
  payoutHoldReason: string | null;
}
```

Add this new interface right after CafePayoutBreakdownItem's type alias (after the existing line `export type CafePayoutBreakdownItem = ...`):

```
export interface CafePayoutDestinationSummary {
  upiVpa: string | null;
  bankAccountNumberMasked: string | null;
  bankIfsc: string | null;
  accountHolderName: string | null;
  payoutAccountId: string;
  payoutAccountVersion: number;
  updatedAt: string;
}
```

```
export interface RevealedCafePayoutDestination {
  upiVpa: string | null;
  bankAccountNumber: string | null;
  bankIfsc: string | null;
  accountHolderName: string | null;
  payoutAccountId: string;
  payoutAccountVersion: number;
}
```

Replace `getCafePayoutBreakdown`:

```
export async function getCafePayoutBreakdown(
  cafeId: string,
): Promise<{ bookings: CafePayoutBreakdownItem[]; destination:
CafePayoutDestinationSummary | null }> {
  return call(() => apiClient.get(`/api/v1/admin/cafe-
payouts/${cafeId}/breakdown`));
}

export async function revealCafePayoutDestination(cafeId: string):
Promise<RevealedCafePayoutDestination> {
  return call(() => apiClient.post(`/api/v1/admin/cafe-payouts/${cafeId}/reveal-
destination`));
}
```

Replace `createCafePayout`:

```
export async function createCafePayout(
  cafeId: string,
  body: {
    utrReference: string;
    paymentMethod: string;
    destinationType: 'upi' | 'bank';
    expectedPayoutAccountVersion: number;
    confirmedPaymentMade: true;
    notes?: string;
    proofImageUrl?: string;
    adminNote?: string;
    paidAt?: string;
  },
): Promise<{ payout: CafePayout }> {
  return call(() => apiClient.post(`/api/v1/admin/cafe-payouts/${cafeId}`, body));
}
```

- ☐ **Step 3: Type-check**

Run: `cd frontend && npx tsc --noEmit -p tsconfig.json` Expected: errors at every existing caller of `createCafePayout/getCafePayoutBreakdown` that doesn't yet

supply the new required fields — this is expected and resolved in Task 10, which is the only caller of these functions in the whole frontend (confirmed: `admin/cafe-payouts/page.tsx` is the sole consumer).

- ☐ **Step 4: Commit**

```
git add frontend/src/lib/api/admin.ts frontend/src/lib/api/errors.ts
git commit -m "feat(admin): extend payout API types for destination reveal +
staleness check"
```

hasBank added to the outstanding-list type, a CafePayoutDestinationSummary type for the breakdown response, a revealCafePayoutDestination function, and createCafePayout's body now requires destinationType, expectedPayoutAccountVersion, and confirmedPaymentMade. The one existing consumer (the payables page) is updated in the next commit."

Task 10: Frontend — rebuild the payables modal on the shared `Modal` component

Files:

- Modify: `frontend/src/app/(admin)/admin/cafe-payouts/page.tsx`

Interfaces:

- Consumes: `Modal` from `@/components/ui` (existing), everything from Task 9's `admin.ts`, `isApiError/API_ERROR_CODES` from `@/lib/api/errors` (or `@/lib/api`).
- ☐ **Step 1: Replace the bespoke modal markup and add destination/staleness state**

Replace the imports at the top of the file:

```
'use client';

import { useState, type ChangeEvent } from 'react';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { Banknote, RefreshCw, ChevronRight, Info, Eye, Loader2 } from 'lucide-react';
import {
  listOutstandingCafePayouts,
  getCafePayoutBreakdown,
  createCafePayout,
```

```

uploadPayoutProof,
setCafePayoutHold,
listCafePayoutHistory,
revealCafePayoutDestination,
type AdminOutstandingCafePayout,
type CafePayout,
type RevealedCafePayoutDestination,
} from '@lib/api/admin';
import { isApiError, API_ERROR_CODES } from '@lib/api/errors';
import { queryKeys } from '@hooks/queries/keys';
import { Card, CardContent, Button, Badge, SkeletonCard, ErrorState, EmptyState,
Tooltip, Modal } from '@components/ui';

```

Replace the component's state block (everything from `const [selectedCafeId, ...` through `const [proofUploadError, ...]`):

```

export default function AdminCafePayoutsPage() {
  const queryClient = useQueryClient();
  const [selectedCafeId, setSelectedCafeId] = useState<string | null>(null);
  const [utrReference, setUtrReference] = useState('');
  const [paymentMethod, setPaymentMethod] = useState('neft');
  const [destinationType, setDestinationType] = useState<'upi' | 'bank'>('upi');
  const [notes, setNotes] = useState('');
  const [paidAt, setPaidAt] = useState(() => new Date().toISOString().slice(0,
10));
  const [adminNote, setAdminNote] = useState('');
  const [proofImageUrl, setProofImageUrl] = useState('');
  const [isUploadingProof, setIsUploadingProof] = useState(false);
  const [proofUploadError, setProofUploadError] = useState<string | null>(null);
  const [confirmedPaymentMade, setConfirmedPaymentMade] = useState(false);
  const [revealed, setRevealed] = useState<RevealedCafePayoutDestination | null>
(null);
  const [isRevealing, setIsRevealing] = useState(false);
  const [revealError, setRevealError] = useState<string | null>(null);

```

- ☐ **Step 2: Update the breakdown query and add the reveal + close handlers**

Replace the `breakdownQuery` block and add a `closeModal` helper right after it:

```

const breakdownQuery = useQuery({
  queryKey: [...queryKeys.admin.all, 'cafe-payouts', 'breakdown',
selectedCafeId],
  queryFn: () => getCafePayoutBreakdown(selectedCafeId as string),
  enabled: !!selectedCafeId,
});

function closeModal() {
  setSelectedCafeId(null);
  setRevealed(null);
  setRevealError(null);
  setConfirmedPaymentMade(false);

```

```

    setDestinationType('upi');
  }

  async function handleReveal() {
    if (!selectedCafeId) return;
    setRevealError(null);
    setIsRevealing(true);
    try {
      const result = await revealCafePayoutDestination(selectedCafeId);
      setRevealed(result);
    } catch (err) {
      setRevealError((err as Error)?.message ?? 'Failed to reveal payout
destination.');
```

- ❑ **Step 3: Update the create mutation**

```

const createMutation = useMutation({
  mutationFn: () =>
    createCafePayout(selectedCafeId as string, {
      utrReference,
      paymentMethod,
      destinationType,
      expectedPayoutAccountVersion:
breakdownQuery.data?.destination?.payoutAccountVersion ?? 0,
      confirmedPaymentMade: true,
      notes: notes || undefined,
      proofImageUrl,
      adminNote: adminNote || undefined,
      paidAt: paidAt ? new Date(paidAt).toISOString() : undefined,
    }),
  onSuccess: () => {
    closeModal();
    setUtrReference('');
    setNotes('');
    setAdminNote('');
    setProofImageUrl('');
    setPaidAt(new Date().toISOString().slice(0, 10));
    queryClient.invalidateQueries({ queryKey: [...queryKeys.admin.all, 'cafe-
payouts'] });
  },
  onError: (err) => {
    if (isApiError(err) && err.code === API_ERROR_CODES.PAYOUT_DESTINATION_STALE)
{
      setRevealed(null);
      breakdownQuery.refetch();
    }
  },
});
```

- ❑ **Step 4: Replace the modal markup**

Replace the entire `{selectedCafeId && selectedCafe && ( <div className="fixed inset-0 ..." > ... </div> )}` block with a `Modal`-based version. The list of café rows above it, and the `HoldToggle/PayoutHistoryTable` functions below it, are unchanged.

```
{selectedCafe && (
  <Modal
    isOpen={!selectedCafeId}
    onClose={closeModal}
    title={selectedCafe.cafeName}
    size="lg"
    footer={
      <div className="flex flex-col gap-2">
        {createMutation.isError && (
          <p className="text-xs text-error">
            {(createMutation.error as Error)?.message ?? 'Failed to record
payout.'}
          </p>
        )}
        {selectedCafe.payoutOnHold && (
          <p className="text-xs text-error">
            Payouts to this café are on hold: {selectedCafe.payoutHoldReason
|| 'no reason given'}.
          </p>
        )}
        <Button
          variant="primary"
          fullWidth
          disabled={
            !utrReference.trim() ||
            !proofImageUrl ||
            isUploadingProof ||
            createMutation.isPending ||
            !selectedCafe.payoutDestinationSubmitted ||
            selectedCafe.payoutOnHold ||
            !confirmedPaymentMade
          }
          onClick={() => createMutation.mutate()}
        >
          {createMutation.isPending ? 'Recording...' : `Mark ₹
${selectedCafe.outstandingAmount.toFixed(2)} as Paid`}
        </Button>
      </div>
    }
  >
    <div className="flex flex-col gap-4">
      <HoldToggle cafe={selectedCafe} onChange={() =>
breakdownQuery.refetch()} />

      <div>
        <span className="text-caption font-semibold text-text-
secondary">Outstanding</span>
        <div className="font-heading text-h1 text-text-primary">₹
{selectedCafe.outstandingAmount.toFixed(2)}</div>
      </div>
```

```

        {breakdownQuery.data && (
          <div className="max-h-48 overflow-y-auto rounded-xl border border-
border divide-y divide-border">
            {breakdownQuery.data.bookings.map((b) =>
              b.type === 'adjustment' ? (
                <div key={b.adjustmentId} className="flex justify-between px-3
py-2 text-xs bg-error/5">
                  <span className="text-error">Adjustment · {b.reason}</span>
                  <span className="font-bold text-error">₹{b.amount.toFixed(2)}
</span>
                </div>
              ) : (
                <div key={b.bookingId} className="flex justify-between px-3 py-
2 text-xs">
                  <span className="text-text-secondary">{b.bookingReference} ·
{b.sessionDate}</span>
                  <span className="font-bold text-text-primary">₹
{b.ownerSettlementAmount.toFixed(2)}</span>
                </div>
              )
            )}
          </div>
        )}

    <div className="flex flex-col gap-2 p-3 rounded-xl border border-border
bg-surface-hover">
      <div className="flex items-center justify-between gap-2">
        <span className="text-xs font-semibold text-text-secondary">Payout
destination</span>
        <Tooltip content="Submitted by the owner – KHEL-0 cannot
independently verify a UPI ID or bank account. Double-check this looks right before
sending money.">
          <Info className="h-3.5 w-3.5 text-text-tertiary cursor-help" />
        </Tooltip>
      </div>
      <div className="text-caption">
        UPI: <span className="font-mono text-text-primary">
{breakdownQuery.data?.destination?.upiVpa || 'not set'}</span>
      </div>
      <div className="text-caption">
        Bank: <span className="font-mono text-text-primary">
{revealed?.bankAccountNumber
  ? `${revealed.bankAccountNumber} / IFSC ${revealed.bankIfsc}`
  : breakdownQuery.data?.destination?.bankAccountNumberMasked
  ? `${breakdownQuery.data.destination.bankAccountNumberMasked}
/ IFSC ${breakdownQuery.data.destination.bankIfsc}`
  : 'not set'}
      </span>
      </div>
      {!revealed &&
breakdownQuery.data?.destination?.bankAccountNumberMasked && (
        <Button variant="ghost" size="sm" isLoading={isRevealing} onClick=
{handleReveal} className="gap-1.5 self-start">
          <Eye className="h-3.5 w-3.5" />
          Show full details to pay
        </Button>
      )}
      {revealError && <p className="text-xs text-error">{revealError}</p>}
    </div>
  </div>
)

```

```

</div>

<label className="flex flex-col gap-1">
  <span className="text-xs font-semibold text-text-secondary">Paying
via</span>
  <select
    value={destinationType}
    onChange={(e) => setDestinationType(e.target.value as 'upi' |
'bank')}}
    className="h-10 px-3 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
  >
    <option value="upi">UPI</option>
    <option value="bank">Bank transfer</option>
  </select>
</label>

<div className="flex flex-col gap-3 pt-2 border-t border-border">
  <label className="flex flex-col gap-1">
    <span className="text-xs font-semibold text-text-secondary">UTR /
Reference Number</span>
    <input
      type="text"
      value={utrReference}
      onChange={(e) => setUtrReference(e.target.value)}
      className="h-10 px-3 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
      placeholder="e.g. UTR2024090712345"
    />
  </label>
  <label className="flex flex-col gap-1">
    <span className="text-xs font-semibold text-text-secondary">Payment
Method</span>
    <select
      value={paymentMethod}
      onChange={(e) => setPaymentMethod(e.target.value)}
      className="h-10 px-3 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
    >
      <option value="neft">NEFT</option>
      <option value="upi">UPI</option>
      <option value="imps">IMPS</option>
    </select>
  </label>
  <label className="flex flex-col gap-1">
    <span className="text-xs font-semibold text-text-secondary">Payment
Date</span>
    <input
      type="date"
      value={paidAt}
      onChange={(e) => setPaidAt(e.target.value)}
      className="h-10 px-3 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
    />
  </label>

```

```

        <label className="flex flex-col gap-1">
          <span className="text-xs font-semibold text-text-secondary">Payment
Proof (required)</span>
          <input
            type="file"
            accept="image/jpeg,image/png,image/webp"
            onChange={handleProofFileChange}
            disabled={isUploadingProof}
            className="text-caption text-text-primary file:mr-3 file:h-9
file:px-3 file:rounded-xl file:border file:border-border file:bg-surface-hover
file:text-caption file:font-semibold"
          />
          {isUploadingProof && <span className="text-xs text-text-
secondary">Uploading...</span>}
          {proofImageUrl && !isUploadingProof && (
            <a href={proofImageUrl} target="_blank" rel="noopener noreferrer"
className="text-xs text-primary underline">
              Proof attached – view
            </a>
          )}
          {proofUploadError && <span className="text-xs text-error">
{proofUploadError}</span>}
        </label>
        <label className="flex flex-col gap-1">
          <span className="text-xs font-semibold text-text-secondary">Admin
Note (optional)</span>
          <textarea
            value={adminNote}
            onChange={(e) => setAdminNote(e.target.value)}
            className="px-3 py-2 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
            rows={2}
          />
        </label>
        <label className="flex flex-col gap-1">
          <span className="text-xs font-semibold text-text-secondary">Notes
(optional)</span>
          <textarea
            value={notes}
            onChange={(e) => setNotes(e.target.value)}
            className="px-3 py-2 rounded-xl border border-border bg-surface
text-caption text-text-primary focus:outline-none focus:ring-2 focus:ring-
primary/40"
            rows={2}
          />
        </label>

        <label className="flex items-start gap-2 text-xs text-text-
secondary">
          <input
            type="checkbox"
            checked={confirmedPaymentMade}
            onChange={(e) => setConfirmedPaymentMade(e.target.checked)}
            className="mt-0.5"
          />
          <span>I confirm this payment was made externally using the
destination shown above.</span>
        </label>

```

```
        </div>
      </div>
    </Modal>
  )}
```

- ☐ **Step 5: Type-check**

Run: `cd frontend && npx tsc --noEmit -p tsconfig.json` Expected: no errors.

- ☐ **Step 6: Build**

Run: `cd frontend && npx next build` Expected: succeeds, `/admin/cafe-payouts` compiles.

- ☐ **Step 7: Manual verification in-browser**

Using the `claude-in-chrome` tooling (or a manual local run of `npm run dev`), verify against a seeded dev café with an outstanding balance:

1. Open a payable — the modal is centered, has a visible close button, and its content scrolls internally rather than growing past the viewport at a resized-down (e.g. 700px tall) browser window.
2. Press Escape — the modal closes.
3. Click the dimmed backdrop — the modal closes.
4. Re-open it, click "Show full details to pay" (only visible if the café has bank details) — the full account number appears; closing the modal and reopening it shows it masked again (not cached).
5. Fill in UTR + proof + tick the confirmation checkbox — "Mark ₹X as Paid" becomes enabled only once every required field, including the checkbox, is filled.

- ☐ **Step 8: Commit**

```
git add "frontend/src/app/(admin)/admin/cafe-payouts/page.tsx"
git commit -m "fix(admin): rebuild payables modal on the shared Modal component"
```

The bespoke fixed-position div had no max-height/scroll on its outer card, no backdrop-click-to-close, and no Escape handler — on a short viewport its close/submit buttons could scroll off-screen entirely. The shared components/ui/Modal.tsx already solves all of this correctly. Also adds the destination-reveal button, a paying-via UPI/bank selector feeding the new required destinationType, and a required 'payment was made externally' confirmation checkbox distinct from the UTR field."

Task 11: Full regression verification

Files: none (verification only)

- ☐ **Step 1: Run the full backend test suite**

Run: `cd backend && pytest -v` Expected: all tests pass, including every test touched or added across Tasks 1–7.

- ☐ **Step 2: Type-check the frontend**

Run: `cd frontend && npx tsc --noEmit -p tsconfig.json` Expected: no errors.

- ☐ **Step 3: Build the frontend**

Run: `cd frontend && npx next build` Expected: succeeds, all routes compile including `/admin/cafe-payouts` and `/owner/settings`.

- ☐ **Step 4: Manual browser verification of the full payout flow**

Against a seeded dev environment: as an owner, edit payout details (wrong password rejected, correct password succeeds, confirmation email observed via logs/Resend dashboard); as admin, open the same café's payable, reveal the destination, record a payout, confirm the resulting history entry shows the snapshotted destination; as owner again, change the UPI, and confirm the earlier payout's history entry still shows the *old* destination (not the new one).

- ☐ **Step 5: Manual verification of the promotion fix**

As an owner, edit a promotion with a KHELO code and a max-redemptions value set; clear both fields; save; reload the page; confirm both stay cleared rather than reverting.

This task has no commit of its own — it's a gate confirming Tasks 1–10 are all correctly integrated.
